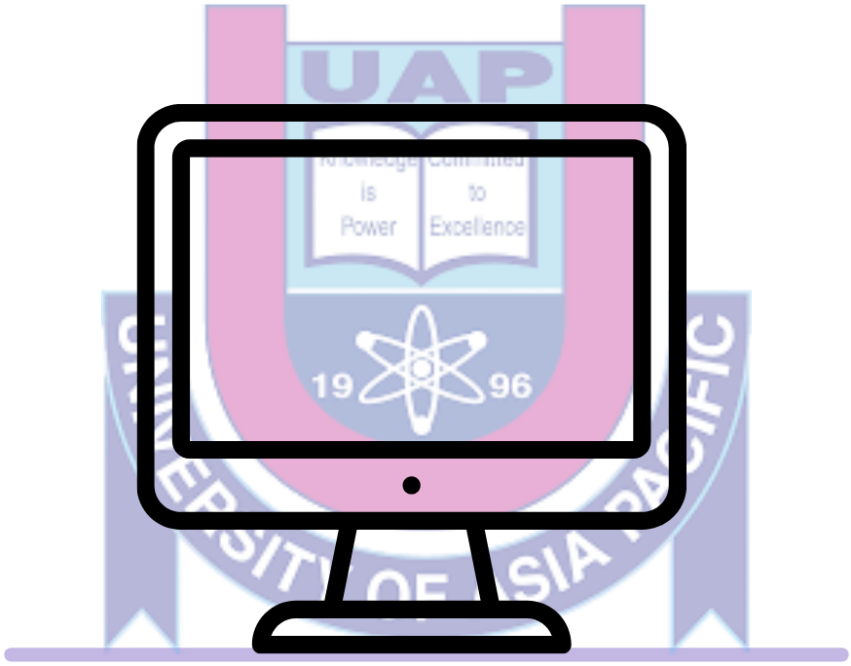
CSE 430



Submitted By:

Mahia Akter Momo

22101122

Sec: C2

Submitted To:

Md Shafayatul Haque

Lecturer

Department of Computer Science & Engineering

University of Asia Pacific

Project Name: MiniLang compiler

Version Control Link -> https://github.com/mahiamOmO/Mini-Lang-Compiler

Slide Link ->

https://www.canva.com/design/DAHKaySHPeA/8XOYksZ8-QuOxS_1_6Edxw/edit

Project Developed By:

Reg ID: 22101122
Section: C2

Project Submitted To:

Md Shafayatul Haque
Lecturer

Table of Contents

1. Project Overview :

MiniLang Compiler is a complete, educational compiler implementation written in C that demonstrates all seven classical phases of compilation. The project provides a hands-on, working implementation of concepts typically taught in compiler design courses, making each phase explicitly visible through formatted terminal output. The compiler accepts programs written in a custom language called MiniLang — a simple, statically-typed imperative language — and processes them through every stage of the compilation pipeline from tokenization all the way to x86-64 assembly generation. The project is hosted publicly on GitHub and is licensed under the MIT License. It is written primarily in C (82%), with supporting Yacc/Bison grammar (12%), Shell scripts (3.9%), Flex lexer rules (1.3%), and a Makefile (0.8%).

2. Objectives :

- Demonstrate all seven classical compiler phases in a single, runnable project
- Provide clear, formatted output at each compilation stage for educational visibility
- Support a meaningful subset of an imperative programming language including variables, arithmetic, conditionals, and loops
- Implement the compiler using industry-standard tools: Flex for lexical analysis and Bison for parsing
- Generate real x86-64 assembly as target code output
- Make the project easy to build and run with a single shell command

3. MiniLang Language Features

MiniLang is a simple statically-typed imperative language. The following table summarizes all supported language constructs:

Feature	Syntax	Notes
Data Types	int (integer)	Currently only integer type supported
Variable Declarations	int x;	Must be declared before use
Arithmetic Ops	+ - * / %	All basic math operations
Comparison Ops	== != < <= > >=	Full relational operator set
Conditional	if / if-else	With block bodies { }
Looping	while (cond) { }	Condition-based iteration
Comments	// comment	Single-line style comments

4. Compilation Phases

#	Phase	Input	Output	Tool
1	Lexical Analysis	Source Code	Tokens	Flex
2	Syntax Analysis	Tokens	Parse Tree	Bison
3	Semantic Analysis	Parse Tree	Validated Tree	Bison/Parser
4	Symbol Table	Identifiers	Variable Info	Parser
5	TAC Generation	Parse Tree	Intermediate Code	Parser
6	Code Optimization	TAC	Optimized TAC	Optimizer
7	Code Generation	TAC	x86-64 Assembly	Codegen

4.1 Phase 1 — Lexical Analysis

Implemented using Flex (src/lexer.l), the lexer reads the source code character by character and groups them into tokens. Each recognized token — whether a keyword, identifier, literal, or operator — is displayed with its category and value:

```
[1  ] [LEXICAL] KEYWORD       : int
[2  ] [LEXICAL] IDENTIFIER    : x
[3  ] [LEXICAL] SYMBOL        : ;
```

4.2 Phase 2 — Syntax Analysis

Implemented using Bison (src/parser.y), the parser checks whether the token stream conforms to MiniLang's grammar. A visual parse tree is rendered with ASCII art to show the hierarchical structure of the program:

├─ StatementList

├─ Decl

├─ Type : int

├─ ID : x

4.3 Phase 3 — Semantic Analysis

The semantic analysis phase verifies that the program is meaningful — for example, that variables are declared before use and that operations are applied to compatible types. Results are reported per statement:

- ✓ Declaration statement parsed
- ✓ Assignment statement parsed
- ✓ If statement semantically valid

4.4 Phase 4 — Symbol Table

The symbol table is a data structure that stores information about every declared identifier in the program. It is built during parsing and displayed as a formatted table:

Name	Type	Scope	Category	Value
x	Int	global	Variabel	0

4.5 Phase 5 — TAC Generation

Three-Address Code (TAC) is an intermediate representation where each instruction has at most three operands. It bridges the gap between the high-level parse tree and low-level assembly. Temporary variables (t0, t1, ...) are introduced for sub-expressions:

t0 = a < b

t1 = a + c

t2 = t0 * t1\

4.6 Phase 6 — Code Optimization

The optimizer applies simple transformations to the TAC to improve efficiency — for example, constant folding, elimination of redundant computations, and simplification of trivial expressions. Optimizations applied are reported explicitly:

- ✓ Simple Expression Optimization Applied
- ✓ Constant Folding Applied

4.7 Phase 7 — Code Generation

The final phase translates the optimized TAC into x86-64 assembly instructions. Registers (R1, R2, ...) are used for operand storage, and instructions such as MOV, CMP, SETL, and ADD are emitted:

```
MOV R1, a
MOV R2, b
CMP R1, R2
SETL t0
MOV R1, a
MOV R2, c
ADD R1, R2
```

5. Project Structure :

The repository is organized into a clean directory layout with separate folders for source code, build output, and example programs. The table below describes every file in the project:

File/Path	Language	Description
src/lexer.l	Flex	Defines token patterns via regular expressions
src/parser.y	Bison/C	Grammar rules + all 7 compiler phases logic
src/lex.yy.c	C (generated)	Auto-generated lexer from Flex — do not edit
src/parser.tab.c	C (generated)	Auto-generated parser from Bison — do not edit
src/parser.tab.h	C (generated)	Parser header file with token definitions

main.c	C	Compiler driver — initializes and runs the pipeline
build/minilang	Binary	Compiled executable target
build.sh	Shell	Main build script — runs all 7 phases end-to-end
build-full.sh	Shell	Extended build with additional options
compile.sh	Shell	Standalone compilation script
run-wsl.sh	Shell	WSL-specific runner for Windows users
test.sh	Shell	Automated test runner across all examples
Makefile	Make	Alternative build via make command
examples/*.ml	MiniLang	7 sample source programs for testing

6. Example Programs:

The project ships with seven .ml example programs that exercise different language features and are used for testing all seven compilation phases.

File	Descrition	Key Features
test.ml	Simple variable declarations and assignments	int x; x = 5;
complex.ml	Loops and conditions combined	while, if, expressions
ifelse.ml	If-else conditional branching	if-else blocks
counter.ml	While loop counting	while with counter variable
nested.ml	Nested if-else inside while loop	Complex control flow
Multiif.ml	Multiple nested if statements	Deeply nested conditionals
Sumeven.ml	Sum of even numbers with while loop	Arithmetic + loops

7. Build and Usage

7.1 Prerequisites

- GCC compiler

- Flex lexer generator
- Bison parser generator
- WSL (Windows Subsystem for Linux) — optional, for Windows users

7.2 One-Command Build

The entire compiler can be built and run from source with a single command. The build script automatically invokes Flex, Bison, and GCC in sequence, then runs the compiler against a test input:
bash build.sh
For Windows users running WSL:
wsl bash -c "cd /mnt/c/Users/HP/Downloads/compiler && bash build.sh"

7.3 Build Process Steps

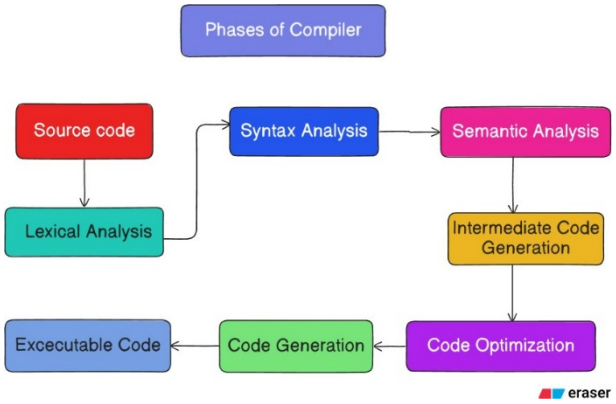
- flex src/lexer.l → generates src/lex.yy.c
- bison -d src/parser.y → generates src/parser.tab.c and src/parser.tab.h
- gcc main.c src/lex.yy.c src/parser.tab.c -o build/minilang
- ./build/minilang < examples/test.ml → runs all 7 phases

7.4 Running Individual Examples

```
cat examples/test.ml    | ./build/minilang # simple declarations
cat examples/complex.ml | ./build/minilang   # loops + conditionals
cat examples/ifelse.ml  | ./build/minilang   # if-else
cat examples/counter.ml | ./build/minilang   # while loop counter
cat examples/nested.ml  | ./build/minilang   # nested control flow
cat examples/sumeven.ml | ./build/minilang   # sum of even numbers
```

8. Architectural Design:

The compiler follows a classic linear multi-pass pipeline architecture. Data flows from the source file through each phase in sequence:



The Bison parser (src/parser.y) acts as the central coordinator — it drives the token consumption from Flex and handles semantic analysis, symbol table management, TAC generation, optimization, and code generation all within a single grammar-action framework. The main.c driver simply opens the input file and calls yyparse() to start the pipeline.

9. Strengths and Limitations :

9.1 Strengths

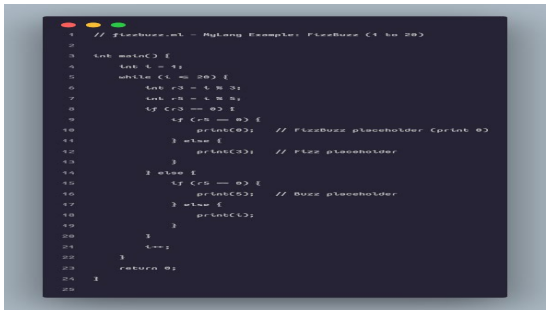
- Covers all seven classical compilation phases in one working project
- Highly educational — each phase prints readable, formatted output
- Uses industry-standard tools (Flex and Bison) matching real-world practice
- Simple single-command build reduces setup friction
- Seven diverse example programs demonstrate the full feature set
- Clean directory structure separates source, build artifacts, and examples
- MIT licensed — free for academic and personal use
- WSL compatibility scripts make it accessible on Windows

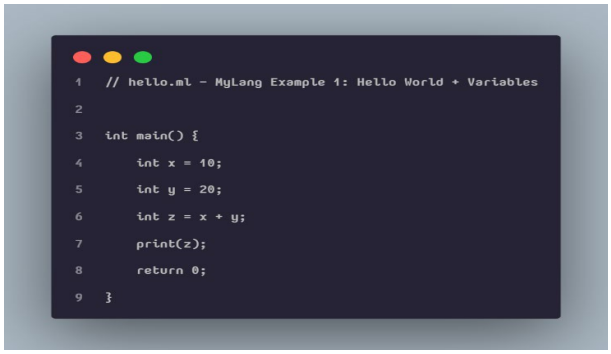
9.2 Limitations

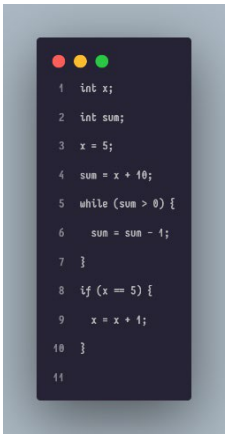
- Only integer (int) is supported as a data type — no float, string, boolean, or user-defined types
- No functions or procedures — the language is flat (no subroutine abstraction)
- No standard input/output statements — programs cannot print values or read user input

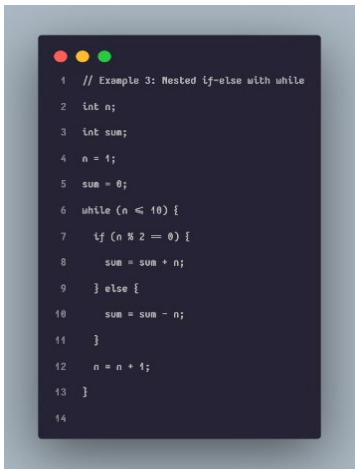
- Code optimization is limited to simple expression-level passes; advanced optimizations (e.g., loop unrolling, dead code elimination) are not implemented
- The x86-64 assembly output uses abstract register names (R1, R2) rather than actual CPU registers, so it cannot be directly assembled or linked
- No error recovery — the compiler stops at the first syntax or semantic error
- The symbol table is flat (global scope only); nested scopes are not modeled

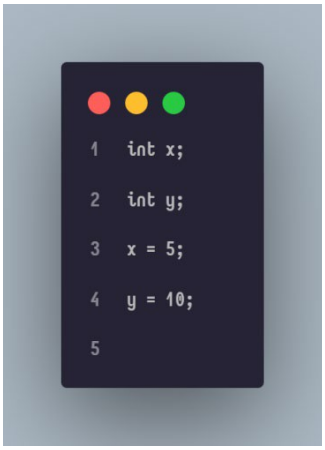
10. Input & Output Sample :

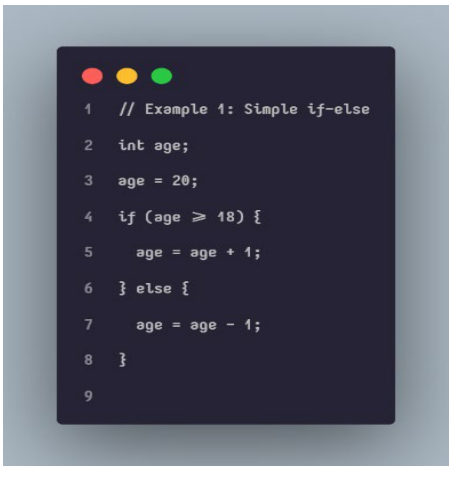


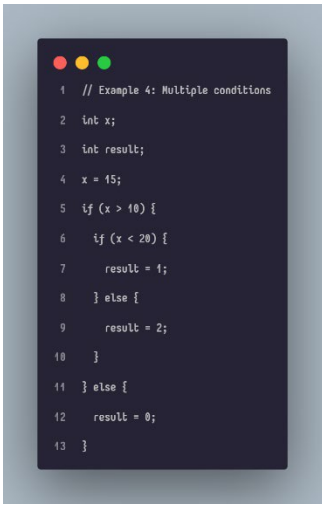












PHASE 7: TARGET CODE GENERATION (x86-64)

```
MOV R1, a
MOV R2, b
CMP R1, R2
SETL t0
MOV R1, a
MOV R2, c
ADD R1, R2
MOV t1, R1
MOV R1, t1
MOV R1, t1
MOV R1, a
MOV R2, 50
CMP R1, R2
SETL t2
MOV R1, a
```

PHASE 4: SYMBOL TABLE

Name	Type	Scope	Category	Value
x	int	global	variable	0
y	int	global	variable	0

PHASE 2: SYNTAX ANALYSIS (Parse Tree)

```
├─ StatementList
  ├─ Decl
    ├─ Type : int
    ├─ ID : x
  ├─ Decl
    ├─ Type : int
    ├─ ID : y
  ├─ Assign
    ├─ ID : x
    ├─ Num : 5
  ├─ Assign
    ├─ ID : y
    ├─ Num : 10
```

PHASE 1: LEXICAL ANALYSIS (Tokenization)

```
Scanning source code for tokens...

  [1  ] [LEXICAL] KEYWORD        : int
  [2  ] [LEXICAL] IDENTIFIER     : x
  [3  ] [LEXICAL] SYMBOL         : ;
  ✓Variable 'x' declared successfully
  [4  ] [LEXICAL] KEYWORD        : int
  [5  ] [LEXICAL] IDENTIFIER     : y
  [6  ] [LEXICAL] SYMBOL         : ;
  ✓Variable 'y' declared successfully
  [7  ] [LEXICAL] IDENTIFIER     : x
  [8  ] [LEXICAL] OPERATOR       : =
  [9  ] [LEXICAL] NUMBER         : 5
  [10 ] [LEXICAL] SYMBOL         : ;
  [11 ] [LEXICAL] IDENTIFIER     : y
  [12 ] [LEXICAL] OPERATOR       : =
```

PHASE 3: SEMANTIC ANALYSIS

✓Declaration statement parsed
✓Assignment statement parsed
✓If statement semantically valid
✓Assignment validation successful
✓While condition semantically valid

PHASE 6: CODE OPTIMIZATION

✓Simple Expression Optimization Applied
✓Simple Expression Optimization Applied

```
        MiniLang Compiler v1.0
     Complete Compilation Pipeline


COMPILATION PHASES:
  [1] Lexical Analysis      (Flex Scanner)
  [2] Syntax Analysis       (Bison Parser)
  [3] Semantic Analysis     (Type Checking)
  [4] Symbol Table          (Variable Storage)
  [5] TAC Generation        (Intermediate Code)
  [6] Optimization          (Code Improvement)
  [7] Code Generation       (Assembly Output)
  [8] Symbol Table Display  (Final Variables)
```

11. Conclusion :

The MiniLang Compiler is a well-structured, educational compiler project that successfully demonstrates all seven phases of the compilation process within a single codebase. By leveraging the industry-standard Flex and Bison tools alongside hand-written C code for semantic analysis and code generation, the project provides a clear and instructive mapping from theory to implementation.

Its strengths lie in its completeness as a teaching tool — every phase is explicit, visible, and accompanied by formatted output that makes the internal workings of the compiler transparent. The variety of example programs and the single-command build further lower the barrier to exploration.

Future enhancements could include support for additional data types, function declarations, I/O statements, more sophisticated optimizations, and linking the generated assembly to a real assembler such as NASM or GAS to produce executable binaries. As it stands, MiniLang Compiler serves as an excellent foundation for learning and extending compiler design concepts.

References

- Repository: https://github.com/mahiamOmO/Mini-Lang-Compiler
- Aho, Lam, Sethi & Ullman — Compilers: Principles, Techniques, and Tools (Dragon Book)
- Flex Manual: https://westes.github.io/flex/manual/
- Bison Manual: https://www.gnu.org/software/bison/manual/
- MIT License: https://opensource.org/licenses/MIT